Platform Architecture

On-ramp, Off-ramp & Multi-currency Accounts

Reference architecture for a payment platform that supports crypto and fiat collection and payouts, multi currency accounts, and multi region expansion from a single codebase.

Core owns money truth. Payout Service owns rails. Launch in Singapore with home_region on all customer data so US and EU markets can be enabled later without rebuilding the platform.

Design canon: BLUEPRINT | NON-NEGOTIABLE | multi-region compliance | launch architecture

1. Design principles (non negotiable)

Single source code, multi region by design

* One source tree, one migration chain, one binary per service for all markets.
* No regional product forks.
* Launch Singapore (home_region = sg); enable US/EU via config, capacity, and licenses.
* home_region on customer scoped data from day one; per region house books.
* Write once, pin data by region, turn markets on with config and capacity.

Money truth

* Core is the only writer of financial state.
* PostJournalEntry is the only ledger write path.
* Decimal money only; append only history; reversals for corrections.
* Balanced journals; idempotent (source, external_ref); available vs posted.

Modular monolith at launch

* Modules as packages with exclusive table ownership and interfaces.
* One CockroachDB database for Core at launch.
* Outbox in the same transaction as state change; Core owns the money saga.

Product model

* Crypto and fiat first class on the same MCA and ledger.
* Profiles: business or person (internal parties only).
* Recipients and payers are external counterparties.
* Public API: /payments, /payouts, /transfers.

2. System architecture

Clients talk to Core API. Core orchestrates ledger, MCA, onboarding, compliance, and transactions. Payout Service integrates banks, crypto custody, and FX. Data ownership is strict.

```
Clients:   Web / Checkout, Mobile, Merchant API
    |
Core API   (auth, rate limits, REST)
    |
Core Service ....................... CockroachDB (+ Redis)
  Ledger, transactions, MCA, onboarding, compliance client
    |
Payout Service ..................... MySQL only
  Bank, crypto, FX adapters
    |
External: Banks / PSPs, Crypto custody, AML vendors
```

Core modules (same deployable)

API, Txn engine, Ledger (PostJournalEntry), Account / MCA, Onboarding (business or person, home_region), Compliance client, Quote / fees, Webhook, Outbox. Packages communicate through interfaces, not shared tables.

Data ownership

* Core API / Core Service -> CockroachDB (ledger and domain state).
* Core API -> Redis (idempotency, cache).
* Payout Service -> MySQL only (rails operations).
* PII documents -> per region object storage (not SQL).

3. On ramp and off ramp (one engine)

One transaction engine serves collection, payout, and internal transfer. Same ledger, same MCA (fiat + crypto), same home_region rules.

On ramp (collection)

* API: POST /v1/payments (hosted or API mode).
* Ledger: pending credit, then post after AML and finality.
* AML: async with hold.
* Rails: deposit / VA / pay in via Payout Service.
* Launch path: in kind crypto (e.g. USDC to USDC), then local currency settlement later.

Off ramp (payout)

* API: POST /v1/payouts then confirm within quote TTL.
* Ledger: debit hold first; post on settle or void on fail/expiry.
* AML: sync, fail closed before dispatch.
* Rails: bank or crypto wallet dispatch via Payout Service.

Internal transfer

* Both sides internal accounts: ledger only (no Payout Service).
* Same home_region required in v1.

4. Multi region (Singapore first)

* home_region set at profile creation; inherited by accounts, ledger, transactions.
* Customer data: REGIONAL BY ROW ready. Reference data (currencies): GLOBAL.
* House books (omnibus, fee, FX, suspense, aml_hold) per region.
* PostJournalEntry rejects mixed region lines.
* Rollout: enable region, seed house accounts, regional buckets/Redis, same image, corridors.

5. Product surface

Three pillars on one core

* Merchant rails: collection and payouts (crypto first, then fiat).
* Business multi currency accounts.
* Consumer multi currency accounts.

Shared ledger, party model (business or person), MCA, transaction engine, and rails. Business KYB first; person schema ready from day one.

Merchant API

* POST/GET /v1/payments: collections
* POST /v1/payouts + confirm: payouts
* POST /v1/transfers: internal
* Balances, transactions, recipients, rates, signed webhooks
* Idempotency Key; decimal string amounts; API keys at launch

Delivery milestones

* M0: Ledger foundation, home_region, MCA, API keys, outbox, region registry (sg).
* M1: Crypto collection in kind, webhooks, minimal recon.
* M2: Crypto payouts, sandbox, checkout, AML.
* M3: Fiat settlement, rate lock, fiat payouts, fx_position.
* M4: Transfers, short/over paid handling, polish.

6. Technology & rails

Stack

Go, CockroachDB (Core), MySQL (Payout Service), Redis, gRPC, Docker/Kubernetes, AWS, observability (Datadog / Prometheus / Grafana).

Integration classes

* Banks: J.P. Morgan, DBS, OCBC, Standard Chartered, Citi, HDFC, SeaBank, Maybank, KBank, and regional banks.
* Mobile wallets & networks: Alipay, WeChat Pay, GrabPay, PayPal, Stripe, MoneyGram, RippleNet.
* FX & cross border: Wise, CurrencyCloud, Thunes.
* Custody & crypto: Fireblocks, BitGo, Copper, Circle, Coinbase Prime, QuickNode.

Sources: BLUEPRINT, NON-NEGOTIABLE principles, multi-region compliance, launch architecture.